Final Term Project | Object-Oriented Programming

OOP Analysis of Rich Python Library

A Comprehensive Case Study on Production-Grade Software Architecture and Design Patterns

MOHSIN ATTA
Roll No: 2061

QAZI REYAN
Roll No: 2050

REHAN BUKHARI
Roll No: 2089

Department of Computer Science & Data Science
The Islamia University of Bahawalpur (IUB)

Library Overview

Rich is a cutting-edge Python library for rendering beautiful, styled text and visuals directly in any terminal window.

- **Console Objects:** High-level abstraction for printing.
- **Layout Engines:** Build responsive terminal dashboards.
- **Syntax Highlighting:** Real-time code formatting.
- **Progress Indicators:** Async spinners and progress bars.

Why Analyze Rich?

Production-Grade

Moving beyond simple "Animal -> Dog" textbook examples to code used by thousands of professionals globally.

The Four Pillars

Provides one of the cleanest implementations of Inheritance, Encapsulation, Polymorphism, and Abstraction.

Open Source

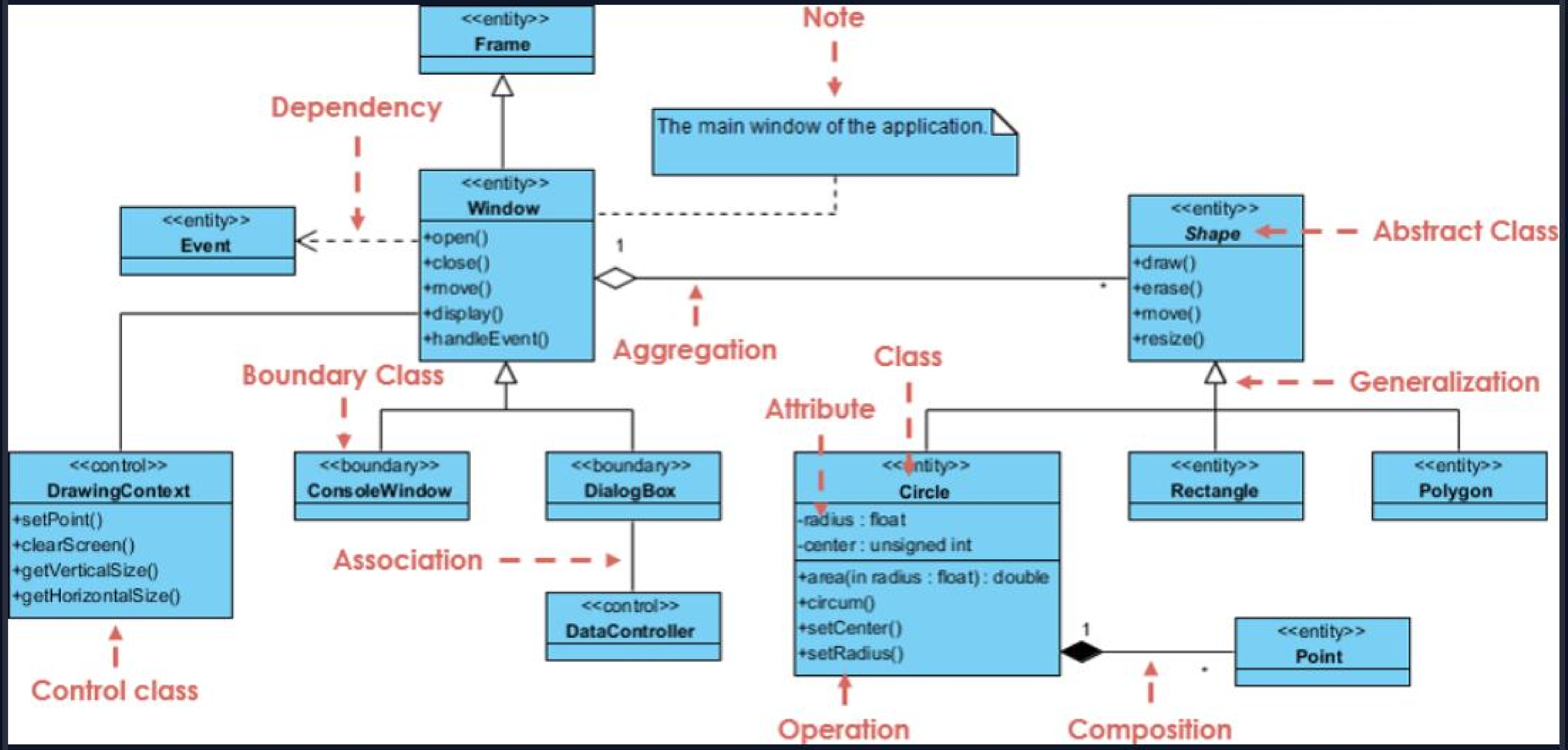
A transparent codebase that allows us to trace architectural design decisions directly on GitHub.

Architectural Foundation

The Class Hierarchy

The library is built on a robust skeletal structure that prioritizes reuse and scalability:

- JupyterMixin: The root base class providing core environment detection and rendering logic.
- Child Specialization: Tables, Panels, and Trees all inherit from this root.
- ABC Layer: Abstract classes like RenderHook enforce implementation rules for extensions.



Encapsulation in Action

```
class Progress:
  def __init__(self):
    self._tasks = {} # Private attr

  @property
  def tasks(self):
    return list(self._tasks.values())
```

Managed State Access

Rich shields sensitive internals—such as concurrent execution locks and task maps—from direct modification.

- Uses leading underscore (_) to indicate internal states.
- Exposes read-only access via @property decorators.
- Ensures thread safety during background rendering loops.

Deep Inheritance Hierarchy

Base Class

ProgressColumn

Defines the baseline geometry, padding, and refresh logic shared by all column types.

Specialized Child

BarColumn

Inherits core logic but overrides the `render()` method to draw the physical progress bar.

Functional Reuse

SpinnerColumn

Reuses 90% of the parent class code, focusing only on the unique animation frames.

The Console Protocol

Behavorial Polymorphism

The Console doesn't need to know *what* it's printing. It only cares that the object follows a specific protocol.

- Unified Interface: The __rich_console__ magic method.
- One Method Call: `console.print()` yields different visual results based on object type.
- Loose Coupling: Objects define their own rendering, making the library highly extensible.

```
# Multiple objects, one interface
console.print(my_table)
console.print(my_panel)
console.print(my_progress)
```

Structural Abstraction

Defining Contracts

Uses Python's `abc` module to define structural nodes that manage layout calculations.

Enforcing Rules

Subclasses of `ProgressColumn` MUST implement `render()` or the system will fail at instantiation.

RenderHooks

Abstract interface allows custom rendering plugins to integrate cleanly into the internal loop.

Architectural Trade-offs

"Rich avoids global states. Explicit context injection (passing the Console instance) ensures thread isolation at the cost of more verbose method signatures."

Case Study: StudentDashboard

Extending the Library

To prove scalability, we built StudentDashboard which extends the base `Table` class:

- **Direct Inheritance:** Gains all border and color logic for free.
- **Custom Logic:** Encapsulates status formatting (Red/Green) internally.
- **Composition:** Nested inside a `Panel` for clean UI enclosure.

```python
class StudentDashboard(Table):
  def add_task(self, name, done):
    color = "green" if done else "red"
    self.add_row(name, task, color)
```

Production vs. Academic OOP

Concept	Standard Textbook Example	The Rich Library Approach
Inheritance	Animal -> Dog (Simple Is-A)	Mixin-based (JupyterMixin)
Encapsulation	Private variables for hiding	Property decorators for safe access
Polymorphism	Different Speak() methods	The `__rich_console__` Protocol
Abstraction	Shape drawing interfaces	Enforcing UI rendering rules via ABCs

Any Questions?

Rich demonstrates that OOP is not just theory; it's the backbone of professional-grade tools.

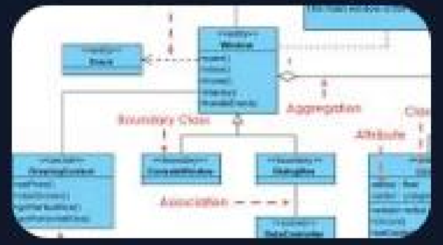
Research Team

Mohsin Atta | Qazi Reyan | Rehan Bukhari

Image Sources

https://miro.medium.com/v2/resize:fit:1400/1*ocmhnXCDek4lPKXAGflSxA.png

Source: bugfixxer.medium.com



https://cdn-images.visual-paradigm.com/guide/uml/uml-class-diagram-tutorial/18-uml-class-diagram-example-gui.png

Source: www.visual-paradigm.com

https://crystalpng.com/wp-content/uploads/2024/10/IUB-logo.png

Source: crystalpng.com

https://abanoubhanna.com/img/visual-studio-dark.jpg

Source: abanoubhanna.com